

16/07/2026

1614

Maximum Nesting Depth of the
Parentheses

Dry Run

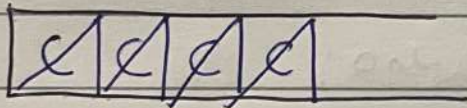
→ Better → TC → $O(n)$

→ SC → $O(n)$ & Stacky

S → "(1+(2*3)+((8)14))+1"

(I)

maxDepth → 0, 1, 2, 3



cnt → 0, 1, 2, 3

1, 2, 3, 4, 0

1 condition:-

if (ch == '(') {

cnt the →

cnt++;

upcoming

st.push(ch);

open

maxDepth = Math.max(maxDepth,

braces

1) calculate the maxDepth. cnt);

push the
'('

2 condition:-

else if (ch == '(')
else {

cnt--;

~~st.push~~

st.pop();

}

⇒ So after the for loop
↳ maxDepth = 3

⇒ Code:-

int helper (String s) {

int maxDepth = 0;

int cnt = 0;

~~Stack~~ Stack <Character> st = new Stack<>();

for (int i = 0; i < s.length(); i++) {

char ch = s.charAt(i);

if (ch ~~==~~ == '(') {

cnt++;

st.push(ch);

maxDepth = Math.max(maxDepth, cnt);


```

    }
    else if (ch == ')') {
        cnt--;
        st.pop();
    }
}

```

```

} return maxDepth;
}

```

}

Note → (1) Here we know the input is already VPS & Valid Parentheses String.

→ Stack is for Validation

→ cnt is for depth.

(2) So we only use cnt to optimize the Sc.

TC → O(n)

Dry Run Optimized Sc (1)

Step " (1 + (2 * 3) + ((8) / 4)) + 1 "

(I) - - - - - last iteration

int maxDepth → 0 1 2 3

int cnt → 0 1 2 3 2 1 0

① condition

if (ch == '(') {

cnt++;

~~maxDepth~~

maxDepth = Math.max(maxDepth, cnt);

② condition

else if (ch == ')') {

cnt--;

}

Code

~~TC~~ TC $\Rightarrow O(n)$ SC $\Rightarrow O(1)$

int helperOptimized(String S)

int maxDepth = 0

int cnt = 0;

for (int i = 0; i < S.length(); i++) {

char ch = S.charAt(i);

if (ch == '(') {

~~cnt++~~ cnt++;

maxDepth = Math.max(maxDepth, cnt);

}

else if (ch == ')') {
 cnt --;

}

} return maxDepth;

}